

Contents

1	Introduction	3
2	What is added to vanilla FLASH?	3
3	More on build/compile process	4
4	SETTINGS IN BRIDGE MULTIPLES	4
5	Hydro	4
5.1	CFL and pressure holes	4
5.2	Artificial viscosity	4
6	Gravity	5
6.1	Unsplit Gravity Calculation	5
6.2	Sink particle gravity	5
6.3	Static Grav Potential	6
6.3.1	Joung, Ibanez-Mejia implementation	6
7	Gravity Bridge	6
8	The Sub-Cycle: rays, ionization, heat/cool	7
9	Ray tracing	7
10	Ionization	8
10.1	Ionization Terms	8
10.2	Ionization Equilibrium	9
11	Heating/Cooling	9
11.1	Gas heat/cool	9
11.1.1	Photo-heating	10
11.1.2	CR heating	10
11.1.3	Photoelectric heating	10
11.1.4	Atomic cooling curve	12
11.1.5	Molecular cooling curve	13
11.1.6	Collisional gas-to-dust cooling	13
11.2	Dust heat/cool	14
11.3	Banerjee/Seifried mol+dust module	15
11.4	JCIM??: phenHeat/IGWCurve	15
11.5	Joung/Baczynski?: phenHeat with rad xfer	15
11.6	Joung/Baczynski?: phenHeat	15
12	Stellar winds	17
13	Supernova Driving	17
13.1	Wall: momentum+thermal SN driving	17
13.2	Joung: pure thermal SN driving	18
13.3	Particle stuff in Joung unit	20
13.4	Compare to other SN driving schemes	20
13.5	Superbubble schemes and behavior	22
14	Code development	22
14.1	Pull Requests	23

15 Lessons Learned, Cheatsheets, and FAQ	23
15.1 FLASH GOTCHAS!!!!	23
15.2 FLASH runtime parameters	24
15.3 FLASH ifdef variables	24
15.4 FLASH variable cheat sheet	24
15.4.1 FLASH field variables	24
15.4.2 FLASH particle variables	24
15.4.3 FLASH species variables	25
15.5 Frequently asked questions	25

1 Introduction

Torch is a star formation simulation code with magnetohydrodynamics (FLASH), radiation transfer (FERVENT), self-gravity & sinks, star particles, feedback (stellar winds, SNe from SeBa tracks), and N-body dynamics (AMUSE).

The **Torch Quick Start Guide** is exactly what it sounds like.

A **Torch Manual** (ATM) (1) holds developer knowledge, (2) explains code features in more depth than published literature, and (3) provides historical context to show how **Torch** has evolved. ATM gives references and equations for code features, but should generally not explain physics.

Help improve ATM at: https://bitbucket.org/aarontran/torch_manual

ISM physics references:

- Stahler & Palla 2004 Star Formation
- Draine 2011 Physics of ISM/IGM
- Klessen & Glover 2014 Physical Processes in the ISM, [arXiv:1412.5182](#), [Springer PDF](#).

2 What is added to vanilla FLASH?

Torch is distributed in two repositories. One is `flash4.2.2-rad` which contains MANY add-ons and units accumulated from the mid-2000s to present. The second is `flash4.5-rad` which presents a smaller set of these changes required to run **Torch**.

Bouchut split solver. Christoph Federrath has some comments on how to use Bouchut solver to use. <http://flash.uchicago.edu/pipermail/flash-users/2014-May/001459.html>

“Mtmod” is called at restart by `IO_readCheckpoint.F90`. This is a custom edit to handle RNG state dump/read. **TODO 2019 April 25 make a unit for this**

`Grid_checkGridVar.F90` is located in `flash4.2.2-rad/source/Grid` or `flash4.5-rad/source/Grid_checkVar/`. This file is used to help report where fields are going wonky. In `flash4.2.2-rad`, hooks for grid var debugging are included in:

```
Driver/DriverMain/Unsplit/Driver_evolveFlash.F90
Driver/DriverMain/Unsplit/Couple_AMUSE/Driver_evolveFlash.F90
Driver/DriverMain/Split/Couple_AMUSE/Driver_evolveFlash.F90
```

In `flash4.5-rad`, hooks for grid var debugging are included in:

```
Driver/DriverMain/Unsplit/Couple_AMUSE/Driver_evolveFlash.F90
```

To use this functionality, specify `--defines=DEBUG_GRIDVAR` in your `setup` call. Modify the Driver file to report on whatever grid variable you desire. **TODO 2019 April 29. In the `Grid_checkVar` module, provide example Driver files with the hooks and a README file.**

In `Driver_computeDt` the guardcell filling is changed by josh due to an error of zero dens on restarts...

New unit 2019 July: AT adding `source/numericalTools/Random` to prevent constantly rewriting code for random number generation. Also allows user to choose a seed!

Josh’s november commit - hybrid riemann solver. https://bitbucket.org/joshuawall/flash4.5/commits/bc708aa9d27dfc04chg-source/physics/Hydro/HydroMain/unsplit/hy_uhd_dataReconstOneStep.F90

“Added the ability for the code to mark regions where data reconstruction falls back to 1st order as ”shocked” regions so that when using the HYBRID Reimann solver the solver will also fall back to HLL instead of HLLD or HLLC.”

Crit

which requires an edit to `hy_uhd_dataReconstOneStep.F90` . . .

3 More on build/compile process

AMUSE: I (Aaron) use a commit from October 2018, 369471e994c06ea39fbd530881c59973b112d6fd. Later commits from Jan 2019 gave weird MPI errors. Cause uncertain.

AMUSE: we have run into problems with OpenMPI C++ bindings. This is an issue for new versions of OpenMPI (4.x); you may have to build OpenMPI from source with a configure flag to request bindings. See:

- <https://github.com/amusecode/amuse/pull/366>
- <https://github.com/joshuawall/amuse/commit/7ed7f2e45679587a43cb7b1f172297160f9bb820>

AMUSE in August 2019 (after HDF5 configure update) doesn't work quite right... first you need to change the configure call for with-hdf (see `./configure --help`). Then, need to manually fix `HDF5_LIBS` in `config.mk`. See Aaron's notes, 2019 August 13.

FLASH with OpenMPI v4.0.0 may throw errors like:

```
undefined reference to 'mpi_type_extent_'
undefined reference to 'mpi_address_'
undefined reference to 'mpi_type_struct_'
undefined reference to 'mpi_type_hvector_'
```

Resolution was (early 2019) to use OpenMPI nightly build with `'-enable-mpi1-compatibility'`. Now it should be fixed in 4.0.1. To address, see <https://github.com/open-mpi/ompi/issues/6114> For OpenMPI v4.0.0, you may get references

HDF5 building: place to check how HDF5 was configured is

```
hdf5-1.10.5/lib/libhdf5.settings
```

under whatever your HDF5 build directory is, the top-level dir containing bin, lib, include, et cetera.

4 SETTINGS IN BRIDGE MULTIPLES

- Choose `cloud_mass`, or equivalently your IMF high-mass cutoff, manually.
- Adjust `sink_rad` to match your NXB setting, should be 16 for NXB=16.

5 Hydro

Remember to fix small numbers – the default values are NOT correct.

Unsplit solver references: <https://ui.adsabs.harvard.edu/abs/2009JCoPh.228..952L/abstract> <https://ui.adsabs.harvard.edu/abs/2011ApJ...737...13K/abstract>

MHD code comparison: <https://ui.adsabs.harvard.edu/abs/2011ApJ...737...13K/abstract>

5.1 CFL and pressure holes

A problem that has recurred through work with Juan, Josh, etc. Bouchut, unsplit, and `posdefdt` are supposed to help alleviate. Helpful thread that summarizes the problem <http://flash.uchicago.edu/pipermail/flash-users/2019-May/002908.html>

5.2 Artificial viscosity

About 0.1 is good. See Fryxell paper section 3.2 for brief description, I am unsure if the Lapidus paper actually matches whatever is in the code.

Only turns on when shocks are present. For example, using the 1-D advection equation for just density:

$$\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x} (\rho u) = 0$$

where ρu is the flux, the artificial viscosity looks something like

$$\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x} \left(\rho u - C \frac{\partial u}{\partial x} \frac{\partial \rho}{\partial x} (\Delta x)^2 \right) = 0$$

where C is the dimensionless runtime parameter `avisc`, and the viscosity is only turned on in cells satisfying $\partial u / \partial x < 0$; i.e., zones that are steepening. This all based on reading `unsplit/hy_uhd_getFaceFlux.F90`

It looks consistent with artificially smearing the flux, EXCEPT that you only keep the cross-term for some reason . . . why this exact form?

$$\frac{\partial^2}{\partial x^2} (\rho u) = \rho \frac{\partial^2 u}{\partial x^2} + u \frac{\partial^2 \rho}{\partial x^2} + 2 \frac{\partial \rho}{\partial x} \frac{\partial u}{\partial x}$$

6 Gravity

6.1 Unsplit Gravity Calculation

The unsplit MHD solver includes gravity in `hy_uhd_unsplit.F90` as follows. Hydro is evolved first in each timestep, so other evolution subroutines (like particle advance) can use the updated gravity potential and accelerations.

```
! compute GPOT_VAR from Poisson equation via Gravity_potentialListOfBlocks(...)
! also compute sink->gas accel SGA{X,Y,Z}_VAR and gas->sink accel ACC{X,Y,Z}_PART_PROP
call hy_uhd_prepareNewGravityAccel(blockCount,blockList,...)

do i = 1,blockCount
  ! compute grav{X,Y,Z} accels via Gravity_accelOneRow(...),
  ! accel is SGA{X,Y,Z}_VAR plus finite difference of GPOT_VAR
  call hy_uhd_putGravityUnsplit(blockID,...,gravX,gravY,gravZ)

  ! update velocities and energy from grav{X,Y,Z}
  call hy_uhd_addGravityUnsplit(blockID,...,gravX,gravY,gravZ,...)
end do
```

Note: in normal FLASH, GPOT_VAR includes contribution from massive non-sink particles via PDEN_VAR. This is disabled in **Torch** since all massive non-sink particles are handled by the AMUSE bridge.

QUESTION: are sink particles double counted in grav accel? They require the `active/massive/Leapfrog` integrator, which defines both `MASS_PART_PROP` and `PDEN_VAR`. Therefore, cursory reading of code suggests that sinks are included twice:

- first in `Gravity_potentialListOfBlocks`: GPOT includes sink mass via PDEN, and
- again in `Particles_sinkAccelGasOnSinksAndSinksOnGas`: computes sink acceleration upon gas directly.

ANSWER: no, we are OK.

- In default FLASH: in `Particles_updateGridVar`, which would map prt masses to PDEN, a special exclusion is made for sink particles. Only non-sink massive prtls are included in PDEN.
- In Torch: all massive prtls excluded from GPOT.

6.2 Sink particle gravity

In `Particles_sinkAdvanceParticles` sink particles are advanced using both gas- ζ sink and sink- ζ sink accelerations. The gas- ζ sink acceleration `ACCX,Y,Z_PART_PROP` is not re-computed, but just taken and assumed up-to-date, probably set in Hydro. The sink- ζ sink acceleration is computed by direct calculation according to FLASH manual, so not super efficient.

6.3 Static Grav Potential

6.3.1 Joung, Ibanez-Mejia implementation

Use flash4.2.2-rad with BHTree, but not BHTree4.3. The FLASH source code is directly modified in:

flash4.2.2-rad/source/physics/Gravity/GravityMain/Poisson/Gravity_accelOneRow

to hook in a static grav acceleration. But, BHTree4.3 will override this. All AMUSE couplings in flash4.2.2-rad will also override.

7 Gravity Bridge

NOTE: the coupling code subroutines:

Gravity_getAccelAtPoint.F90
Gravity_getPotentialAtPoint.F90

are not currently in use. Josh used these in an older version of bridge interface, but all that stuff is commented out now. These routines are not called anywhere in FLASH code. - AT, 2020 May 09.

Pseudocode for stars to kick gas+sinks, AMUSE bridge.

```
FUNCTION get_gravity_particles_on_gas(dt, kick_number)

  if (kick_number == 1) then

    ! gas
    VEL{X,Y,Z}_VAR += BGA{X,Y,Z}_VAR * dt

    BGA{X,Y,Z}_VAR = 0

  else if (kick_number == 2) then

    call Gravity_potentialListOfBlocks(GPOT_VAR)
    ! result: GPOT_VAR computed from DENS_VAR (solve Poisson eqn)
    !       beware, this is NOT default FLASH behavior

    call Gravity_potentialListOfBlocksPDENonly(BGPT_VAR, ACTIVE_PART_TYPE)
    ! result: BGPT_VAR computed from PDEN_VAR (solve Poisson eqn)

    call Gravity_accelListOfBlocks(IAxis, BGAX_VAR, BGPT_VAR)
    call Gravity_accelListOfBlocks(JAxis, BGAY_VAR, BGPT_VAR)
    call Gravity_accelListOfBlocks(KAxis, BGAZ_VAR, BGPT_VAR)
    ! result: BGA{X,Y,Z}_VAR contains {x,y,z}-acceleration from BGPT_VAR

    ! gas
    VEL{X,Y,Z}_VAR += BGA{X,Y,Z}_VAR * dt

  end if

  call Particles_longRangeForce(SINK_PART_TYPE, BGPT_VAR)
  ! result: sink ACC{X,Y,Z}_PART_PROP contains acceleration from BGPT_VAR

  ! sink
  VEL{X,Y,Z}_PART_PROP += ACC{X,Y,Z}_PART_PROP * dt

END FUNCTION
```

Pseudocode for gas+sinks to kick stars, AMUSE bridge.

```
FUNCTION get_gravity_gas_on_particles(dt)

  ! compute gravitational acceleration and potential for particles
```

```

! by CIC mapping field quantities to particle locations and storing values
! in GPOT_PART_PROP, ACCX_PART_PROP, etc. Acceleration comes from
! finite differencing the potential in Gravity_accelListOfBlocks
Particles_longRangeForce(ACTIVE_TYPE_PART, GPOT_VAR)

! call Gravity_accelListOfBlocks(IAXIS, GRAC_VAR, GPOT_VAR)
!   for all blocks:
!       call Gravity_accelOneRow(SWEEP_X, GPOT_VAR)
!       returns: first-order finite difference of GPOT_VAR along x plus SGAX_VAR for a single
!       block
!       result: GRAC_VAR contains x-acceleration from gas (GPOT_VAR) and sinks (SGA{X,Y,Z}_VAR)
!
! call Grid_mapMeshToParticles(...)
!   result: GRAC_VAR mapped to ACCX_PART_PROP via CIC method
!
! repeat for other coordinate directions {JAXIS, KAXIS
!
! result: star ACC{X,Y,Z}_PART_PROP contains acceleration from gas and sinks

! update star velocities
VEL{X,Y,Z}_PART_PROP += ACC{X,Y,Z}_PART_PROP * dt

END FUNCTION get_gravity_gas_on_particles

```

8 The Sub-Cycle: rays, ionization, heat/cool

Moral: in runtime parameters, set the ionization fraction x to the equilibrium value for your initial temperature and density. Cold gas can take $> 10^9$ yr for cosmic ray ionization and recombination to balance; if the initial ionization is wrong, the heating and cooling will be wrong too.

Timestep sub-cycling: only triggered for rapidly varying radiation fields. Tentatively (as of April 2019), heating and cooling is NOT sub-cycled. IF the ionization is stable, **RadTrans** will take two steps. It first computes an ionization timestep using $rt_dt = 0.01 \text{ dt}$. Then, it advances rays, ionization, and heating/cooling using (almost) the full hydro timestep, $rt_dt = 0.99 \text{ dt}$.

Ionization: implicit solve + timestep-estimation is explained in Josh's thesis and Paper II. Baczynski used an explicit solve and accounted for electrons from metals.

TODO low priority (2019 April 25): when the gas is almost fully ionized and $1 - x \ll 1$, the quadratic solve returns x , but the value of $1 - x$ may be inaccurate (subtracting two big numbers to get a small number). Not tested, not yet demonstrated whether this is a problem or not.

TODO low priority (2019 May 01): do the ionization more simply when there are no stars. Don't be clever, don't use Josh's method, just advance the ionization and move on with life!!

9 Ray tracing

The code is split across two units. First a particle unit: **Particles/ParticlesMain/ray_pe** then a rad transfer unit: **physics/RadTrans/RadTransMain/RayRad**. Two PPDEFINE flags indicate presence of ray tracing units: **FERVENT** and **PE_HEATING**. Both ray and ray_pe versions of unit will define **FERVENT**.

There exists an old version of the raytracing particle unit, **Particles/ParticlesMain/ray**, that HAS been modified by Josh; I do not know whether it is up-to-date or works at all.

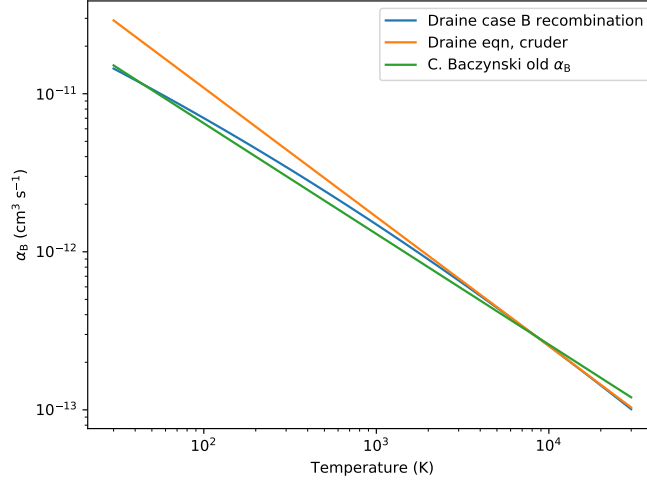


Figure 1. Recombination case B coefficients.

10 Ionization

Torch solves an ionization equation for a pure hydrogen gas:

$$\frac{dx}{dt} = C_{\text{cl}}(T)n_{\text{H}}x(1-x) + k_{\text{ion}}(1-x) - \alpha_B(T)n_{\text{H}}x^2 \quad (1)$$

or equivalently

$$\frac{dn_e}{dt} = C_{\text{cl}}(T)n_en_{\text{HI}} + k_{\text{ion}}n_{\text{HI}} - \alpha_B(T)n_en_p$$

in `calc_ionization.F90`. The ionization fraction $x = n_e/n_{\text{H}}$, the total H nucleon density $n_{\text{H}} = n_{\text{HI}} + n_e$, and the electron/proton densities n_e and n_p are equal. Compared to **FERVENT**, **Torch** omits electrons from metals, dust-catalyzed recombination, and CR ionization of molecules.

10.1 Ionization Terms

Collisional ionization follows equation (3) of Cox and Tucker <http://adsabs.harvard.edu/abs/1969ApJ...157.1157C>. Draine’s textbook restates and abridges Cox/Tucker’s result.

$$C_{\text{cl}} = \frac{1.3 \times 10^{-8} \times 0.83 \times 1 \text{ cm}^3 \text{ s}^{-1}}{13.598^2} T^{1/2} \exp\left(-\frac{13.598 \text{ eV}}{k_{\text{B}}T}\right) \quad (2)$$

$$= 5.8 \times 10^{-9} \text{ cm}^3 \text{ s}^{-1} \left(\frac{T}{10^4 \text{ K}}\right)^{1/2} \exp\left(-\frac{13.598 \text{ eV}}{k_{\text{B}}T}\right) \quad (3)$$

The “Case B” recombination coefficient is from Draine’s book; it is good for $T = 30$ to $30,000$ K:

$$\alpha_B = 2.54 \times 10^{-13} \text{ cm}^3 \text{ s}^{-1} \left(\frac{T}{10^4 \text{ K}}\right)^{-0.8163-0.0208 \ln(T/10^4 \text{ K})}$$

C. Baczynski uses a simpler expression that is credited to Osterbrock (1989). The Osterbrock and Draine expressions differ by $\sim 20\%$ or less over the temperature range given by Draine.

$$\alpha_B = 2.59 \times 10^{-13} \text{ cm}^3 \text{ s}^{-1} \left(\frac{T}{10^4 \text{ K}}\right)^{-0.7}$$

The photoionization rate k_{ion} combines (1) background cosmic-ray (CR) ionization of *molecular* H₂ to H₂⁺, and (2) ionizing photons from FUV-emitting stars. The CR ionization is from eqn (27) and Table 4 of Padovani,

Galli, and Glassgold 2009A&A...501..619P. Josh uses “W98” spectrum shown in Figure 13. **WARNING/TODO:** the exponential cutoff for $N(H_2) > 10^{25} \text{ cm}^{-2}$ is NOT IMPLEMENTED. For that, see the paper’s corrigendum: 2013A&A...549C...3P.

$$k_{\text{ion,CR}} = \zeta_k^{H_2} = 2 \times 10^{-17} \text{ s}^{-1} \left(\frac{N(H_2)}{10^{20} \text{ cm}^{-2}} \right)^{-0.021}$$

To convert from volume density to column density, Josh uses a local approximation valid for $N(H_2) \sim 10^{20}$ to 10^{25} cm^{-2} :

$$N(H_2) \approx \lambda_J n_H$$

$$\lambda_J = \sqrt{\frac{5\pi k_B T}{\mu \rho G}} \propto c_s t_{\text{ff}}$$

where $\mu = \rho/n$ is mean particle mass.

TODO 2019 April 25: column density approximation, do we need to halve the number density from FLASH? Relates to assumptions on molecular vs atomic gas fraction. Very unimportant here: the CR ionization rate is almost constant with respect to number density. But may matter elsewhere.

TODO 2019 May 31: Walch+ 2015 (SILCC) mention: “at no point do we assume that hydrogen is in collisional ionization equilibrium: its contribution to the cooling rate is determined self-consistently at all temperatures, allowing us to properly model the non-equilibrium effects that can strongly affect the cooling rate at temperatures close to 10^4 K ”. **DO WE NEED TO SWEAT THIS? CHECK THE REFERENCES**

10.2 Ionization Equilibrium

To find the correct starting ionization for your simulation, set $dx/dt = 0$. This gives a quadratic for the equilibrium value of x :

$$0 = (C_{\text{cl}}(T) + \alpha_B(T)) n_H x^2 + (-C_{\text{cl}}(T) + k_{\text{ion}}) x - k_{\text{ion}} \quad (4)$$

Beware, the solution is not so accurate for $x \approx 1$. I find unphysical $x > 1$ when solving this quadratic with $\rho = 10^{-25} \text{ g cm}^{-3}$ and $T = 10^6 \text{ K}$. It is probably necessary to swap variable to $1 - x$ to do the calculation accurately.

11 Heating/Cooling

Like ionization, heat/cool is intimately tied to radiation transfer. Location: `HeatMain/HeatCool/phenHeat/mol_and_dust/solver`. Some lessons:

- Breaks with `tradmin = 0`, must be at least 10 Kelvin.

Developer warning: lots of global variables used here.

Truth table for heating/rad parameters:

useHeat	rt_useRadTrans	rt_heatInRad	
F	...	...	Heat.F90 does nothing
T	F	F	Heat.F90 calls RadHeat
T	F	T	Heat.F90 calls RadHeat
T	T	F	Heat.F90 calls RadHeat
T	T	T	Heat.F90 does nothing, RadTrans calls RadHeat

There is also the parameter `useRadTransfer`, which toggles the code for **ray tracing** in `ParticlesMain/ray` and `ParticlesMain/raype`; therefore `useRadTransfer` does not directly affect heating/cooling in the above truth table! (but of course if you turn off stars’ radiation and alter ionization, yes, that will change behavior)

11.1 Gas heat/cool

obeys the equation

$$\frac{d\epsilon}{dt} = \Gamma_{\text{phen}} + \Gamma_{\text{CR}} + \Gamma_{\text{PE}} - \Lambda_{\text{atom}} - \Lambda_{\text{mol}} - \Lambda_{\text{g} \rightarrow \text{d}} \quad (5)$$

where ϵ is specific energy in units erg g^{-1} , so all Γ and Λ have units $\text{erg s}^{-1} \text{g}^{-1}$.

Method: use implicit finite-difference scheme. Let $f(\epsilon) = d\epsilon/dt$. Then, the scheme is:

$$\frac{\epsilon^{n+1} - \epsilon^n}{\Delta t} = f(\epsilon^{n+1}, n^n, T_{\text{dust}}^{n+1}, \dots)$$

where n and $n+1$ index hydro timesteps or a heat/cool subcycle of size Δt . The code seeks the root of:

$$\epsilon^{n+1} - \Delta t f(\epsilon^{n+1}, \dots) - \epsilon^n = 0$$

Developer warning: the bracket- and root-finding code is called as follows:

```
call zbrac(f, x1, x2, xold, dt, success)
call zero(a, b, a_old, machep, t, f, dt)
```

These functions do NOT operate on function **f**, but rather operate on the left-hand side of the equation:

```
x - dt * f(dt, x) - xold = 0
```

in order to implement the implicit solve. This is hugely misleading, but works correct. At least part of the reason, is that Fortran functions are not first-class functions; we cannot easily pass them around like objects. So engineering the bracket- and root- finding code in a general way is a bit non-trivial. Maybe modern Fortran features can alleviate... I am not sure.

Numerics warning: implicit solve is formally stable, but can still be inaccurate. See Townsend <http://adsabs.harvard.edu/abs/2009ApJS..181..391T> for a discussion and an alternative scheme (not implemented in Torch, yet).

11.1.1 Photo-heating

Γ_{phen} is “direct” radiation heating (also called photo-heating), **PHHE_VAR/DENS_VAR**.

11.1.2 CR heating

Γ_{CR} is heating from cosmic rays, eqn (27) and Table 4 of Padovani, Galli, and Glassgold [2009A&A...501..619P](#).

$$\Gamma_{\text{CR}} = (20 \text{ eV}) (2 \times 10^{-17} \text{ s}^{-1}) \left(\frac{N(H_2)}{10^{20} \text{ cm}^{-2}} \right)^{-0.021} = 6.41 \times 10^{-28} \text{ erg s}^{-1} \left(\frac{N(H_2)}{10^{20} \text{ cm}^{-2}} \right)^{-0.021}$$

where 20 eV is the average energy liberated by CR ionization. This should **TODO:check** account for secondary ionizations as well.

11.1.3 Photoelectric heating

Γ_{PE} is photoelectric heating; i.e., “indirect” radiation heating from **PEFL_VAR** liberating electrons on dust grains. First, we need the FUV flux:

$$G = \text{PEFL_VAR} + G_0 f_{\text{shield}}(\rho, n, T) e^{-|z|/z_0, \text{FUV}},$$

where G and G_0 are normalized to the Habing flux $1.6 \times 10^{-3} \text{ erg s}^{-1}$. Josh adopts $G_0 = 1.69$ following Draine (1978). Note: Josh’s thesis defines G_0 as the dimension-ful Habing flux. Here, $G_0 = \text{he_Gzero} = 1.69$ means a heating rate $1.69 \times 1.6 \times 10^{-3} \text{ erg s}^{-1}$; Bakes/Tielens (1994) use our convention, Weingartner/Draine use G instead of G_0 . **TODO (20190619): Weingartner/Draine is the saner convention, and I think Josh is trying to follow this. Change code variable name.**

The heating rate is from Weingartner and Draine [2001ApJS..134..263W](#), equation (44) and Table 2 with $R_V = 3.1$ and $b_c = 6 \times 10^{-5}$.

$$G_f = G \sqrt{T}/n_e$$

$$\epsilon = \frac{7.64 + 4.52 \times T^{0.132}}{1 + 0.04371 G_f^{0.452} (1 + 0.00557 G_f^{0.675})}$$

$$\Gamma_{\text{PE}} = 10^{-26} \text{ erg s}^{-1} \times \epsilon G n / \rho$$

The factor $n/\rho = 1/\mu$ gives a specific heating rate. Note that ϵ influences how the radiation energy from G gets divided between thermal energy of gas versus dust; the heating rate for dust is $\Gamma_{\text{PE} \rightarrow \text{dust}} = 10^{-26} \text{ erg s}^{-1} \times (1 - \epsilon)Gn/\rho$, whenever $\epsilon < 1$.

For reasonable temperatures and densities, say $T < 10^4 \text{ K}$ and $n > 10^{-4} \text{ cm}^{-3}$, the factor $\epsilon > 1$ and so dust doesn't get any heating, all the energy goes into the liberated photo-electron. **TODO 2019 June 14** I'm not sure how to understand the $1 - \epsilon$ in the code, one expects energy from FUV flux to be partitioned into dust and free electron (gas) heat. Any energy dumped in dust will radiate away, but that ought to be accounted for in the dust temperature calculation. Problem is ϵ does not lie within $[0, 1]$. **TODO 2019 June 19** I think this is resolved by rescaling from 10^{-26} to 10^{-24} in WD01 pre-factor.

The Weingartner/Draine expression is a bit confusing - the range of values is quite large at the densities we are considering. What is the range of validity? Also, Josh is adding a local column density approx, which counter-balances this effect somewhat. In Josh's implementation, $n_e = xn_{\text{H}}$ is taken from the ionization solver, so it may not account for metals. Mordecai states that there is some uncertainty in getting n_e correct, since **Torch** doesn't follow O, C, etc chemistry.

Joung,MMML (2006), Hill+ (2012), and Hill+ (2018) use a simpler form of this expression:

$$\Gamma_{\text{PE}} = 10^{-24} \text{ erg s}^{-1} \times \epsilon Gn/\rho$$

with $G = G_0 e^{-|z|/z_0, \text{FUV}}$. Joung,MMML (2006) adopt $G_0 = 1.7$ and $\epsilon = 0.05$, yielding $\Gamma_{\text{PE}}/n = 0.85 \times 10^{-25} \text{ erg s}^{-1} (\text{H atom})^{-1}$. Hill+ (2018) adopt a larger fiducial value $\Gamma_{\text{PE}}/n = 3 \times 10^{-25} \text{ erg s}^{-1} (\text{H atom})^{-1}$.

Kim and Ostriker (2017) do FUV from star particles. They make two adjustments: (1) smooth linear turnoff towards high temperature, using their mean molecular mass prescription (a proxy for ionization), (2) a floor on heating at 0.0024.

Josh also implemented (but now disabled) Wolfire 1995 and 2003 prescription. Let's review these in turn.

Wolfire+ 1995 equations (1-2) are:

$$\Gamma = 10^{-24} \text{ erg s}^{-1} \epsilon G_0$$

$$\epsilon = \frac{0.049}{1 + (G_{\text{f}}/1925)^{0.73}} + \frac{0.037 (T/10^4 \text{ K})^{0.7}}{1 + G_{\text{f}}/5000}$$

where the efficiency ϵ is from Bakes & Tielens 1994 and spans 0.001 to 0.1 for a range of G_{f} values.

Wolfire+ 2003 equations (19-20) are nearly the same, but adjusted to higher PAH abundances:

$$\Gamma = 1.3 \times 10^{-24} \text{ erg s}^{-1} \epsilon G_0$$

$$\epsilon = \frac{0.049}{1 + (G_{\text{f}}/1925/\phi_{\text{PAH}})^{0.73}} + \frac{0.037 (T/10^4 \text{ K})^{0.7}}{1 + G_{\text{f}}/5000/\phi_{\text{m}} t_{\text{PAH}}}$$

Wolfire+ 2003 adopt $\phi_{\text{PAH}} = 0.5$ for their "standard model".

OTHER references: Wolfire 1995, Avillez and Breitschwerdt 2004

Question: how do we budget PEH heating to dust vs to gas? I.e., how to explain Josh's $(1 - f)$ factor.

ISRF: Weingartner/Draine quoted $5.33 \times 10^{-14} \text{ erg cm}^{-3}$ for Habing (1968) and $8.93 \times 10^{-14} \text{ erg cm}^{-3}$ for Draine (1978). Is it safe to assume equilibrium reaction?

Goldsmith [2001ApJ...557..736G](#) and Banerjee, Pudritz, Anderson [2006MNRAS.373.1091B](#) do not include PEH/external heating term, because FUV flux is shielded in cores.

Klessen/Glover 2014 review states dust heating in terms of Γ_{ext} , not separating contributions from CR and photo-heating. The specific heating rate is of order $\Gamma_{\text{ext}} \sim 3 \times 10^{-24} \text{ erg s}^{-1} n/\rho$ which is like $10\times$ larger than the PE heating rate, so it's probably OK to neglect the dust photo-heating that goes into gas rather than dust.

If Γ_{ext} is dominated by photo-heating and not CRs, then on the basis of the cited heating rate, I think the use of $(1 - \epsilon)$ in the code can be adjusted by rescaling the Weingartner/Draine expression to $10^{-24} \text{ erg s}^{-1}$, which lowers ϵ by 10-100.

Krumholz's star formation textbook quotes a lower number:

$$\Gamma = 4 \times 10^{-26} G_0 \text{ erg s}^{-1}$$

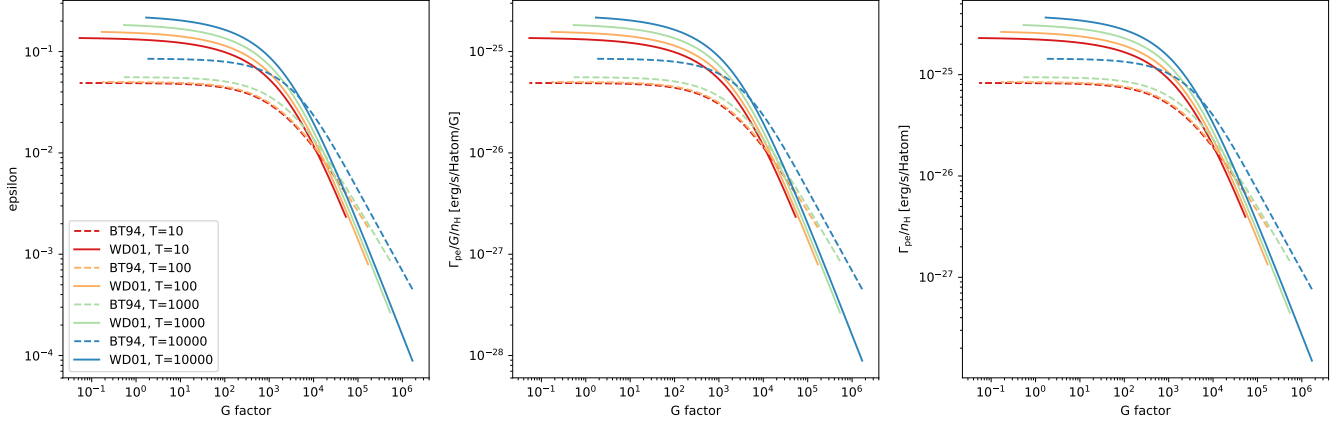


Figure 2. PE heating expressions from BT94 and WD01 Note, I rescaled WD01 expression from $10^{-26} \text{ erg s}^{-1}$ to $10^{-24} \text{ erg s}^{-1}$ in order to permit direct comparison of ϵ with BT94.

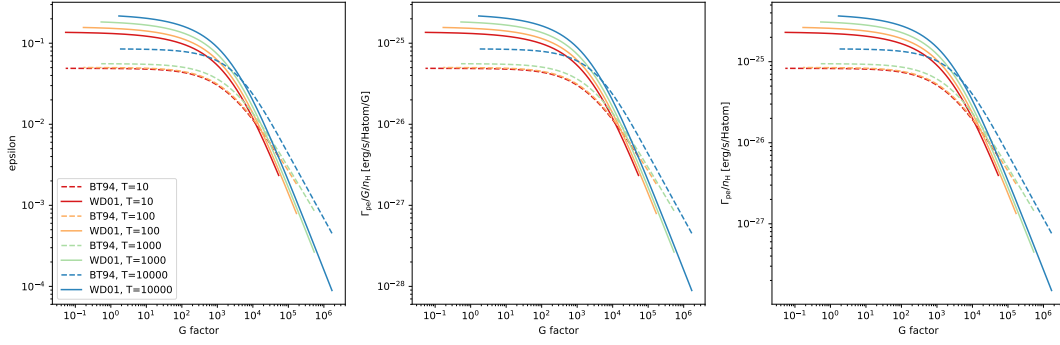


Figure 3. Compare WD01 and BT94 photo-electric heating prescriptions

where I assume Habing is correct for “local interstellar”. For efficient $\epsilon = 0.05$, this implies total heating of gas and dust to be $8 \times 10^{-25} \text{ erg s}^{-1}$. This is quite close to the value used by Joung and M-MML 2006, but somewhat lower than the values considered by Hill+ (2018): 10^{-25} to 10^{-24} (pure PE gas heating, not the total FUV gas+dust heating).

11.1.4 Atomic cooling curve

Λ_{atom} is the cooling curve for atomic gas, originally used in Joung and Mac Low 2006. The references are: Dalgarno and McCray 1972ARA&A..10.375D, Sutherland and Dopita 1993ApJS...88..253S

Dalgarno/McCray include (1) free electron collisions that excite lines in both ions and neutrals, (2) neutral hydrogen collisions in low-ionization regime to excite fine structure lines of C^+ and O. They also state: “The conversion of H to H_2 substantially *reduces* the cooling rate of gas at low temperatures. The spin change mechanism that [controls] fine-structure excitation by neutral atomic hydrogen impact does not operate with molecular hydrogen.” They don’t give rate estimates for H_2 cooling, and suggest just dividing atomic hydrogen rates by 4.

The validity of the atomic cooling curve at lower temperature is uncertain.

Comparison: Wolfire+ 1995 and 2003 explicitly write formulae for the fine-structure line cooling from C II, O I, et cetera.

Dopita & Sutherland is standard: recombination cooling and bremsstrahlung, fewer physical processes need be considered in general.

HEATING AND IONIZATION OF HI REGIONS

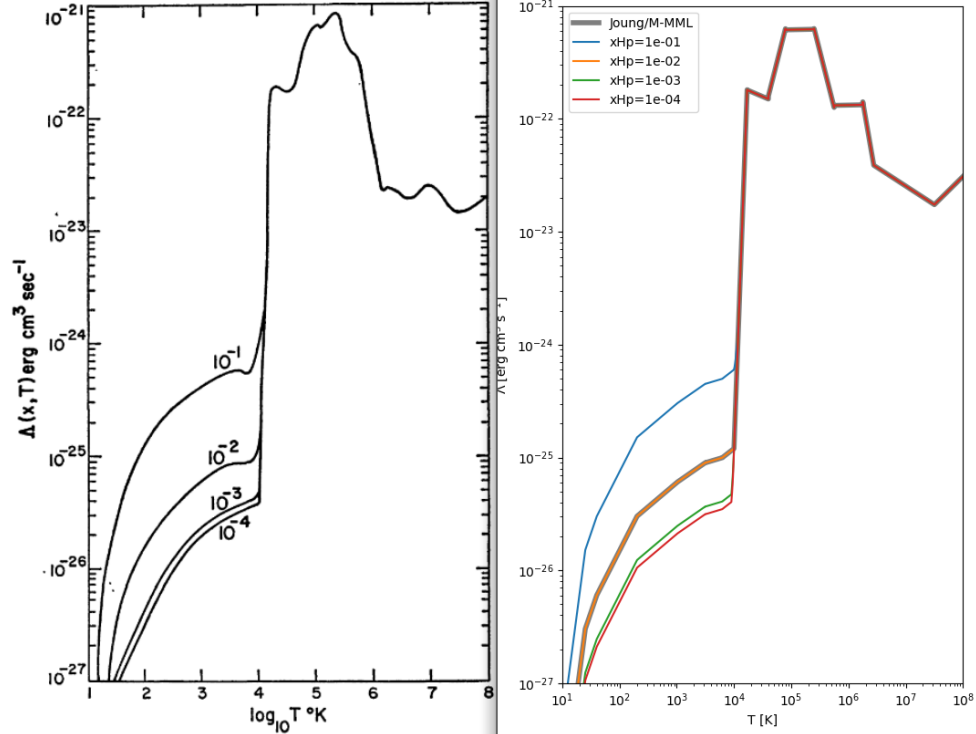


FIGURE 2. The interstellar cooling function $\Delta(x, T)$ for various values of ionization x . The labels refer to the values of x .

Figure 4. Atomic cooling curve compared to Dalgarno/McCray 1972.

11.1.5 Molecular cooling curve

Λ_{mol} uses molecular cooling tables (cool.dat) adopted from Neufeld+ 1993, Neufeld+ 1995; Banerjee papers 2004, 2006a, 2006b, 2015.

TODO: H₂ dissolution and formation timescales are comparable to MC timescales, but we don't account for this chemistry!

Glover & Clark [2012MNRAS.421..116G](#) have simulations with $n \sim 1$ to 100 cm^{-3} with CO forming over ~ 1 Myr. Bigger simulation with colliding flows indicates 1–10 Myr, Clark, Glover, Klessen, Bonnell [2012MNRAS.424.2599C](#)

Liszt [2007A&A...461..205L](#) cites ~ 10 Myr in a neutral H I cloud of column density $N(\text{H}) \sim 4 \times 10^{20} \text{ cm}^{-2}$ (which is low-ish compared to big MW clouds, I think).

Similarly

Useful overview of molecule cooling: Goldsmith and Langer [1978ApJ...222..881G](#)

11.1.6 Collisional gas-to-dust cooling

$\Lambda_{\text{g} \rightarrow \text{d}}$ cooling from gas colliding? with dust, then dust rapidly radiating away (dust acts as a collisional heat sink).

$$\Gamma_{\text{g} \leftarrow \text{d}} = (3.8 \times 10^{-33} \text{ erg cm}^3 \text{ s}^{-1}) \left(1 - 0.8e^{-75/T_{\text{g}}}\right) T_{\text{g}}^{1/2} (T_{\text{g}} - T_{\text{d}}) n_{\text{H}}^2 \rho^{-1}$$

This is erg/s/g instead of erg/s/cm³ expressions quoted in dust heat/cool calculation; they differ by a factor ρ^{-1} .

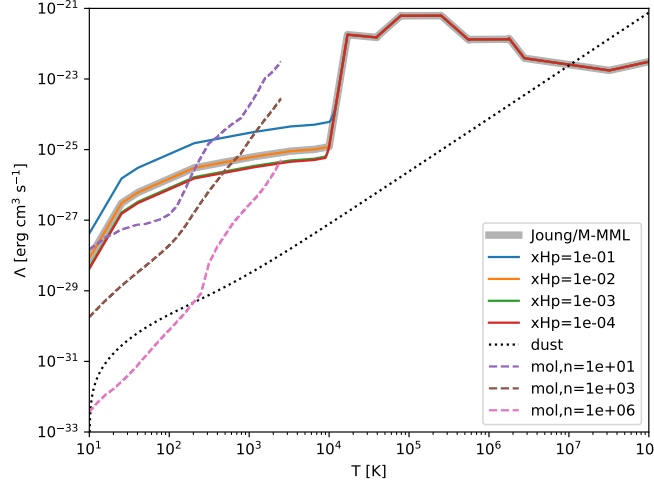


Figure 5. Atomic cooling curve with interpolation at low temperature, and also dust cooling.

11.2 Dust heat/cool

obeys the equation

$$n \frac{d\epsilon}{dt} = \Gamma_{\text{PE}} + \Gamma_{\text{CR}} + \Gamma_{\text{d} \leftarrow \text{g}} - \Lambda_{\text{dust}} \quad (6)$$

where we have abruptly switched to volumetric heating rates, units: $\text{erg s}^{-1} \text{cm}^{-3}$.

Method: to solve for dust temperature, let $f(T_d) = n d\epsilon/dt$. Use Newton’s method. Iteratively compute:

$$T_{\text{d,new}} = T_d - \frac{f(T_d)}{\frac{\partial f}{\partial T_d}}$$

until $T_{\text{d,new}} = T_d$ is converged. This solution method, plus commented out code relating to dust opacity, is described in the appendix of Banerjee, Pudritz, Anderson [2006MNRAS.373.1091B](#). **TODO 2019 May 15: can we just use Brent’s method instead?**

Γ_{PE} and Γ_{CR} are radiative and CR heating respectively. This replaces the commonly assumed “external” heating Γ_{ext} from an assumed interstellar radiation field (ISRF). **TODO 2019 May 14: check that we reproduce correct “external” heating rate with Habing FUV background?**

$\Gamma_{\text{d} \leftarrow \text{g}}$ is collisional heating of dust by gas from Glover/Clark 2012, taken from Hollenbach/McKee. Here T_g and T_d are gas and dust temperatures respectively.

$$\Gamma_{\text{d} \leftarrow \text{g}} = 3.8 \times 10^{-33} \left(1 - 0.8e^{-75/T_g}\right) T_g^{1/2} (T_g - T_d) n_H^2$$

Λ_{dust} is radiative dust cooling from Goldsmith 2001:

$$\Lambda_{\text{dust}} = 6.8 \times 10^{-33} T_d^6 n(\text{H}_2)$$

in units $\text{erg s}^{-1} \text{cm}^{-3}$.

This apparently disagrees with the result stated in Glover and Clark 2012 (reprinted in Klessen and Glover 2014 review). I probably didn’t account for or understand a necessary pre-factor. Anyways, Glover and Clark (2012) state in equations (A7-8):

$$\Lambda_{\text{dust}} = 4\pi\rho_d \int_0^\infty B_\nu(T_d) \kappa_\nu d\nu \approx 4.68 \times 10^{-31} T_d^6 n$$

in units $\text{erg s}^{-1} \text{cm}^{-3}$. Here n is hydrogen number density, dust-to-gas mass ratio is probably 0.01 (they claim it’s a “standard value” but don’t state the number), and opacity $\kappa(\nu)$ is taken from Ossenkopf and Henning [1994A&26A...291..943O](#) for wavelengths $> 1 \mu\text{m}$, and Mathis et al. (1983) for shorter wavelengths. The Ossenkopf/Henning opacities are for non-coagulated grains with thick ice mantles, intended for protostellar core modeling.

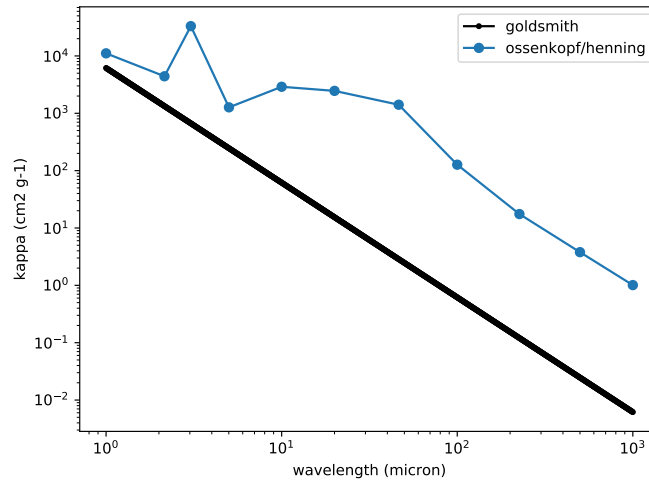


Figure 6. Compare dust opacities from Goldsmith versus Ossenkopf/Henning

The use of molecular versus atomic number density can only explain a factor of 2 difference at most. Why are these values so different?

Answer 2019 May 17: of course, it HAS to be in the opacities. cannot be anywhere else. Next question is which one is correct in which circumstances ... (aaron's guess: MC conditions are quite different from core conditions . . . probably more physical to use goldsmith, which indeed josh is using!!!!)

TODO Big question 2019 May 15: dust cooling equation is only good for $T < 100$ K but we are using it all the way up to $T \sim 10^4$ K. In Draine book and other places, the dust cooling expression plateaus. What gives?

Three papers cited for rates: Goldsmith [2001ApJ...557..736G](#), Glover and Clark [2012MNRAS.421..116G](#), Hollenbach and McKee [1989ApJ...342..306H](#).

11.3 Banerjee/Seifried mol+dust module

Location:

HeatMain/phenHeat/RadHeat/molecules+dust

Code appears to harness FLASH's built-in Cooling unit?

11.4 JCIM??: phenHeat/IGWCurve

Location:

flash4.2.2-rad/source/physics/sourceTerms/Heat/phenHeat

I dunno why this was taken out of HeatMain.

11.5 Joung/Baczynski?: phenHeat with rad xfer

Location:

flash4.2.2-rad/source/physics/sourceTerms/Heat/HeatMain/phenHeat/RadHeat

What is different?

11.6 Joung/Baczynski?: phenHeat

Location:

flash4.2.2-rad/source/physics/sourceTerms/Heat/HeatMain/phenHeat

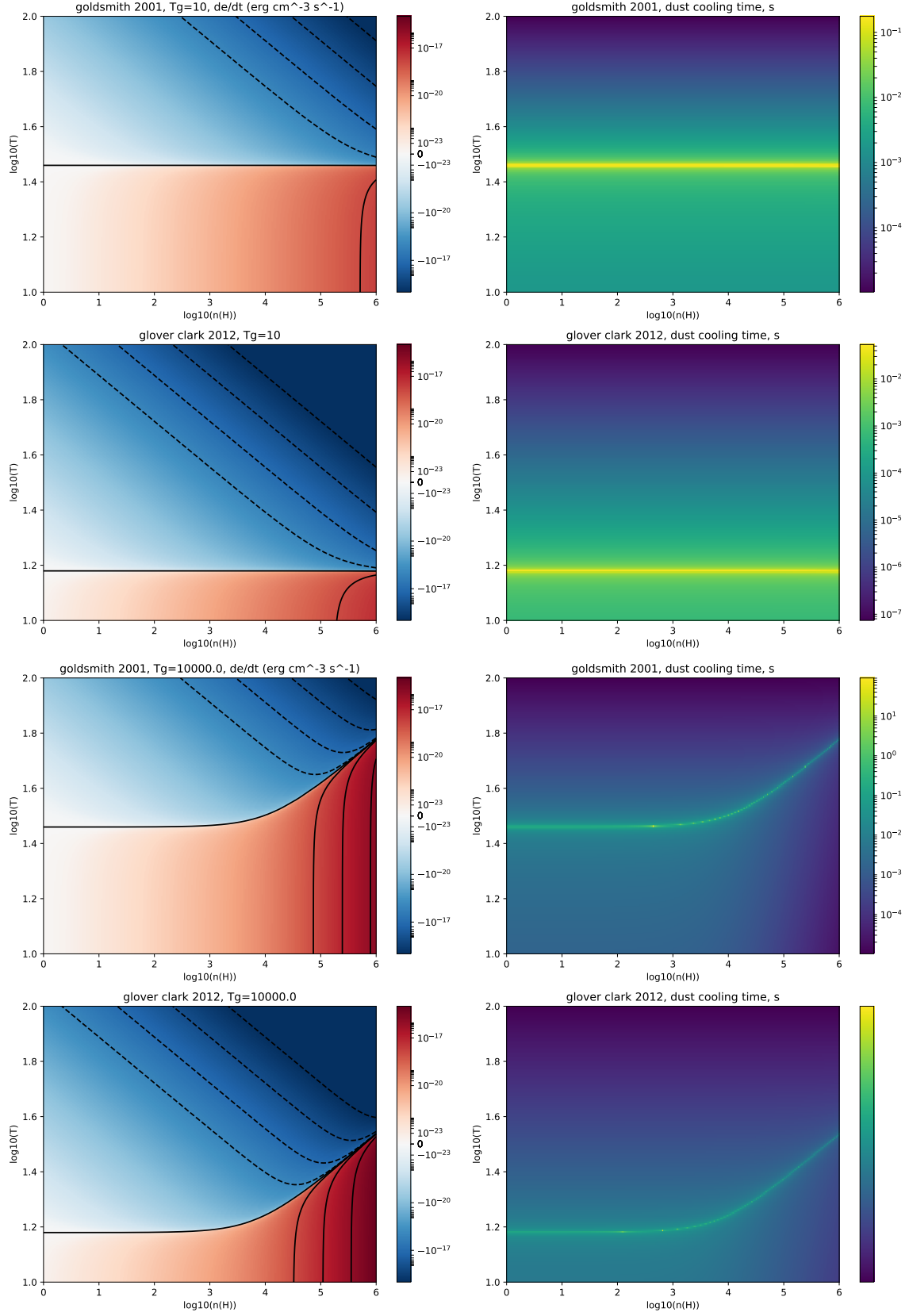


Figure 7. Goldsmith versus Glover/Clark give slightly different dust temperatures. Dust attains equilibrium temperature in a fraction of a second. Left column: net dust heating/cooling rate for given T_{dust} and n_{H} . Right column: timescale for dust heating/cooling. Top four panels, $T_{\text{gas}} = 10$; bottom, $T_{\text{gas}} = 10^4$.

Heating: background FUV, user sets via `he.peheat`, and photo-electric heating via variable `PHHE.VAR`.

Cooling: Ryan Joung wrote a cooling function from some combination of Sarazin (1986), Raymond (1976), and Dalgarno & McCray (1972).

Sub-cycle as follows. During i -th hydro step, compute the cooling time using an explicit method, where n and $n+1$ index the sub-cycle steps within the i -th hydro step.

$$\Delta t_{\text{sub}}^n = \min(f_{\text{sub}} t_{\text{cool}}^n, t_{\text{hy}, i+1} - t_{\text{sub}}^n)$$

$$t_{\text{cool}} = \frac{\epsilon^n}{|d\epsilon/dt|} = \frac{\epsilon^n}{|n^2 \mathcal{L}/\rho|} = \frac{\epsilon^n}{|n\Gamma(\mathbf{r}) - n^2 \Lambda(T^n)|/\rho}$$

With subcycle timestep in hand, advance subcycle energy to ϵ^{n+1} with an explicit method:

$$\frac{\epsilon^{n+1} - \epsilon^n}{\Delta t_{\text{sub}}^n} = n^2 \mathcal{L}/\rho = [n\Gamma(\mathbf{r}) - n^2 \Lambda(T^n)]/\rho$$

12 Stellar winds

`source/Particles/ParticlesMain/active/Sink/Couple_AMUSE/wind`

Star particles have the following important properties:

<code>BGDY_PART_PROP</code>	background gas density at time of wind start
<code>CREATION_TIME_PART_PROP</code>	creation time
<code>DMDT_PART_PROP</code>	star's mass loss rate
<code>POSX_PART_PROP</code>	x position
<code>POSY_PART_PROP</code>	y position
<code>POSZ_PART_PROP</code>	z position
<code>VELW_PART_PROP</code>	star's mass loss rate

which are used to inject winds.

Note: `ref_radius` was formerly used in particle mark refine/derefine. But particle mark refine/derefine criterion has been moved to junk. **TODO (2019 Jul 03) Why was the prtl refinement removed?** Seems like a natural thing to keep around, for all stars that do have wind (criterion: winds turned on, correct stellar type and mass, dm/dt nonzero).

There is an interesting adjustment to the injection scheme. For winds without mass-loading, Josh weights each cell by the solid angle Ω computed from

$$\cos^2(\Omega/4) = \frac{1 + \frac{1}{2} \frac{\Delta^2}{r^2}}{1 + \frac{1}{2} \frac{\Delta^2}{r^2} + \frac{\Delta^4}{r^4}}$$

which should make the mass density decrease as r^{-2} (i.e., constant mass in each shell, as expected for free expansion), according to commit `flash4.2.2-rad 88fc47a`.

Note, the feature for adding thermal energy to momentum-conserving wind is relatively new / not heavily tested. Josh has some comments to this effect, correct amount of thermal energy to add is not 100% clear. Commit is `FLASH4.5 00272ec`.

13 Supernova Driving

13.1 Wall: momentum+thermal SN driving

`source/Particles/ParticlesMain/energyInjection`

13.2 Joung: pure thermal SN driving

Principle: deposit 100% thermal energy in a sphere. Mass of 60 Msun is chosen to prevent unphysical cooling (either at too hot or too cold temperatures).

TODO 2019 May 16 - understand the choice of mass shells, what's going on

References: Joung & Mac Low (2006); Joung, Mac Low, & Bryan (2009); Gatto et al. (?) (2015); Ibáñez-Mejía et al. (2016,2017).

Key units are:

```
source/physics/sourceTerms/SN
source/physics/sourceTerms/SN/SNSBandom
source/physics/sourceTerms/SN/SNSBandom/Clustersink
source/physics/sourceTerms/SN/SNSBandom/SNsink
source/physics/sourceTerms/SN/SNSBandom/SBpeak
source/physics/sourceTerms/SN/SNSBandom/SNejecta
source/physics/sourceTerms/SN/SNSBandom/SNkinetic
source/Particles/ParticlesMain/sb
source/Particles/ParticlesMain/active/Sink/Clustersink
source/Particles/ParticlesMain/active/Sink/SNsink
```

Further units, used with SNSB unit and Juan's stratboxes:

```
source/Particles/ParticlesMain/passive/RungeKuttaThermo
source/Particles/ParticlesMain/passive/MonteCarlo
source/Particles/ParticlesMapping/NoMap
source/Particles/ParticlesMapping/InZone
source/Particles/ParticlesInitialization/NParPerZone
source/Particles/ParticlesInitialization/NParPerZoneMC
```

To use, request `sourceTerms/SN/SNSBandom` and/or one of its sub-units, and the necessary particles unit will be loaded.

A useful problem to test this unit on is `source/Simulation/SimulationMain/SNSBbox`
`source/Simulation/SimulationMain/SNSBbox.AT`

Aaron ported from FLASH4.2.2 to FLASH4.5 on 2019 May 01. The code is deeply interwoven into vanilla FLASH source, but this is probably the cleanest way to do it (avoids copy-paste, and allows the various init/mapping units to be used in the most modular way).

Easiest way to review changes is by diffing `prtl` directory directly (vanilla FLASH vs Torch), but here is a guide to help disentangle the distinct changes since there's a lot going on.

TODO May 2019: only some not all `prtl` functionality tested.

- SN unit is hardcoded into `Driver sourceTerms`, `initSourceTerms`, `computeDt`, `finalizeSourceTerms`.
- `Particles/Particles.h` edited to register new particle types, mapping, and initialization units. Note that `RungeKuttaStatic` is hidden under `ParticlesMain/sb/`.
- `ParticlesMain/passive/Config` edited to register new passive integrators `RungeKuttaThermo`, `MonteCarlo`.
- New `ParticlesMain/passive` particle types `RungeKuttaThermo`, `MonteCarlo`, and `sb/RungeKuttaStatic` require registration.

Define new `Particles/Particles.h` constants `RUNGEKUTTASTATIC`, `RUNGEKUTTATHERMO`, and `MONTECARLO`.

Add stubs:

```
Particles/localAPI/pt_advanceRKstatic.F90
Particles/localAPI/pt_advanceRKthermo.F90
Particles/localAPI/pt_advanceMonteCarlo.F90
```

and register them in `Particles/localAPI/Makefile` so that code knows how to advance `prtl` from different sub-units.

Edit:

`Particles/ParticlesMain/Particles_advance.F90` to add new case logic for `prtl` advance.

Finally, for RUNGEKUTTATHERMO alone, edit:

Particles/ParticlesMain/Particles_data.F90

Particles/ParticlesInitialization/pt_setDataStructures.F90

to get

TODO/BEWARE! Particles_advance is overwritten by AMUSE coupling code... this will probably break passive tracers.

- **ParticlesInitialization** sub-units NParPerZone and NParPerZoneMC require registration.

Define new Particles/Particles.h constants NPPARPERZONE and NPPARPERZONEMC

Add stubs:

Particles/localAPI/pt_initPositionsNParPerZone.F90

Particles/localAPI/pt_initPositionsNParPerZoneMC.F90

and register them in Particles/localAPI/Makefile so that code knows how to initialize prtl from different sub-units.

Edit:

Particles/ParticlesInitialization/Particles_init.F90

Particles/ParticlesInitialization/Particles_initPositions.F90

Particles/ParticlesMain/Particles_data.F90

to add new case logic for prtl init, and to add some config options.

QUESTION: can we implement as "CUSTOM" instead of named sub-units? ANSWER: maybe. But, this looks possibly useful for things other than SN unit.

- **ParticlesMapping** sub-units InZone and NoMap require registration.

Define new Particles/Particles.h constants INZONE and NOMAP

Add stubs:

Particles/localAPI/pt_mapFromMeshInZone.F90

Particles/localAPI/pt_mapFromMeshNone.F90

and register them in Particles/localAPI/Makefile so that code knows how to map prtl from different sub-units. **WARNING** for ParticlesMapping ALSO register in pt_interface.F90, I don't understand why the init/integrate methods don't need to be registered here for subunits to find.....

Edit:

Particles/ParticlesMapping/Particles_mapFromMesh.F90

to add new case logic for prtl mapping.

- Why do we have to register all this stuff in localAPI? Answer: PRTL code has a bunch of sub-units. Even if our simulation doesn't have all sub-units, it must know the methods from all sub-units in order to compile.

The hardcoding into FLASH source is NOT desirable and makes future upgrades more difficult. However, the edits are small and alternatives had their own cons.

Joung/CB SN heating is applied as a heating rate via PHHE_VAR, which does not play nice with the rad xfer (fluxes get cleared in subcycles, but the SN heating rate needs to be applied continuously).

Tentatively, I (AT) have reverted to older direct EINT_VAR and ENER_VAR injection method, which redistributes the mass within the injection sphere. **TODO 20190614** this also breaks momentum conservation, to be addressed by AT.

To use SB particle unit, currently you have to declare particle type **sb** in your problem Config. It appears to work fine with Josh's code. Although, need to make sure that Josh does explicitly address prtl everywhere, and doesn't assume/require only two prtl type anywhere in logic.

Not sure how to make Particle unit advance work nicely with Josh's setup. In flash4.2.2-rad, CB manually registered Particles_advance method... commit frmo like 2013...

13.3 Particle stuff in Joung unit

13.4 Compare to other SN driving schemes

Key point: if you don't resolve the radius where shell formation occurs with at least 3-5 cells, you will not get energy/momentum injection correct. All authors broadly agree on this point.

Remarks: evaporation of shell mass into hot phase might matter and is generally neglected.

Martizzi+ [2016MNRAS.459.2311M](#) SN feedback in local stratified ISM for MW- and ULIRG-like disks using the subgrid model from Martizzi+ [2015MNRAS.450..504M](#)

- Procedure: inject uniform thermal energy and radial momentum into a sphere of radius $2\Delta x$. Use Martizzi+ 2015 equations (9-10) to figure out how much to inject, for forward shock $R = R_{\text{inj}}$:

$$E_{\text{th}}(R, n_{\text{H}}, Z) = E_{\text{th},0} \begin{cases} 1 & , R < R_{\text{c}} \\ (R/R_{\text{c}})^{\alpha} & , R_{\text{c}} < R < R_{\text{r}} \\ (R_{\text{r}}/R_{\text{c}})^{\alpha} & , R_{\text{r}} < R \end{cases}$$

where R_{c} is cooling radius where snow-plough shell forms, and R_{r} is late-time radius where thermal energy becomes roughly constant. For momentum (changing Martizzi's P_r to p_r) use:

$$p_r(R, n_{\text{H}}, Z) = P_0 \begin{cases} (R/R_0)^{1.5} & , R < R_{\text{b}} \\ (R_{\text{b}}/R_0)^{1.5} & , R > R_{\text{b}} \end{cases}$$

where R_0 is scale radius used to extrapolate S-T power law to small radii, and R_{b} is where SNR momentum becomes roughly constant. The hierarchy is $R_0 < R_{\text{c}} < R_{\text{b}} < R_{\text{r}}$. S-T scale < cooling radius < radius where momentum stabilizes < radius where thermal energy stabilizes. See Figure 4 of Martizzi.

Compute mean density, metallicity within injection sphere. Use Martizzi+ 2015 paper fitting functions, equation (11), to determine α , R_{c} , R_{r} , R_0 , R_{b} . Then, use equations (9-10) reproduced above to inject appropriate energy and momentum.

- They sampled densities 0.01, 1, 100, and 10^4 cm^{-3} . with solar metallicity $Z_{\odot}$.
- When SNR cooling radius is well resolved ($R_{\text{inj}} < R_{\text{c}}$) the model “reduces to injecting 10^{51} erg of thermal energy per SN and letting the simulation code explicitly capture the conversion into radial momentum”.
- Note that cooling radius R_{c} is...

$$R_{\text{c}} \approx 14 \text{ pc } (n_{\text{H}}/1 \text{ cm}^{-3})^{-3/7} (E_{\text{tot}}/10^{51} \text{ erg})^{2/7} (Z/Z_{\odot})^{-1/7}$$

based on Cioffi+ (1988) and Thornton+ (1998). In the high-resolution sims, Martizzi resolve R_{c} with at least 50 cells.

Miao Li+ (incl Jerry, Greg) [2015ApJ...814....4L](#) also simulated SNe in uniform and inhomogeneous environments. See Figure 2 for energy, momentum evolution with radius. However, they don't provide any sub-grid model fitting functions.

Gatto+ [2015MNRAS.449.1057G](#) choose R_{inj} to contain $10^3 M_{\odot}$, and require $R_{\text{inj}} \geq 4\Delta x$ (so, in dense environment. Procedure is as follows...

- If SN is resolved, dump 10^{51} erg pure thermal energy. Gas is heated to $8 \times 10^6 \text{ K}$ ($\mu = 1.3m_{\text{P}}$, H and He, ignore lecs) or $3 \times 10^6 \text{ K}$ ($\mu = 0.5m_{\text{P}}$, H and lec plasma, ignore He).
- If SN is unresolved, R_{inj} contains too much mass and would over-cool. Then, inject momentum instead:

$$p_{\text{ST}} = 2.6 \times 10^5 M_{\odot} \text{ km s}^{-1} (E/10^{51} \text{ erg})^{16/17} (\bar{n}/1 \text{ cm}^{-3})^{-2/17}$$

Flatten density within injection region, add velocity, and heat the injection region to 10^4 K .

- They note that result should under-estimate momentum input; during snowplough phase, SNR may continue to gain momentum. Also, lacks strong shocks and hot gas.

Simpson, Bryan, Hummels, JPO [2015ApJ...809...69S](#)

- Injection stencil is 3x3x3, so $r_{\text{inj}} = 1.5\Delta x$ and will put energy into 4x4x4 cells.
- If $4.5\Delta x < R_{\text{PDS}}$ (same as Martizzi R_C), then just dump thermal energy. Otherwise, do:

$$f_{\text{kin}} = 3.97 \times 10^{-6} \mu n_0 R_{\text{PDS}}^7 t_{\text{PDS}}^{-2} \Delta x^{-2} E_{51}^{-1}$$

- To evaluate t_{PDS} and r_{PDS} use equations (17-18), which also encode density and metallicity dependence. For solar metallicity, $R_{\text{PDS}} = 18.5 \text{ pc } E_{51}^{2/7} n_0^{-3/7}$ is quite similar to expression of Martizzi
- Density range: 0.5 to 500 cm^{-3} , they show test of their prescription for $n = 49 \text{ cm}^{-3}$.

Kim & Ostriker 2015ApJ...802...99K

- Inject a small region, $r_{\text{init}} = \Delta x$ and so touch 2x2x2 cells (8 zones)
- $r_{\text{sf}} = 22.6 \text{ pc } E_{51}^{0.29} n_0^{-0.42}$, note that $2/7 = 0.286$ and $-3/7 = -0.428$ so basically the same as all the other authors
- **Particularly useful for discussing r_{init} convergence, which I haven't seen other papers explore**
- Density range: 0.1 to 100 cm^{-3}

Kim & Ostriker TIGRESS 2017ApJ...846..133K use a complex prescription with ST, EJ, MC regimes.

1. Look at spheres from $R_{\text{min}} = 3\Delta x$ to R_{max} , varying R by $\Delta x/2$. For each sphere, compute M/M_{sf} where $M_{\text{sf}} = 1679 M_{\odot} (n_{\text{H}}/1 \text{ cm}^{-3})^{-0.26}$
2. IF $M > M_{\text{sf}}$ at R_{min} , then ST is unresolved. Inject momentum onto grid, using prescription from hi-res sims of SNRs in two-phase medium. No thermal energy, assume all cooled and gone(?).
3. IF $0.027 M_{\text{sf}} < M < M_{\text{sf}}$ at R_{min} , then ST is resolved. Inject 72% thermal and 28% kinetic energy onto grid.
4. IF $M < 0.027 M_{\text{sf}}$ at R_{min} , increase R until we reach $M \geq 0.027 M_{\text{sf}}$. Then do ST injection.
5. IF $M < 0.027 M_{\text{sf}}$ at R_{max} (can't find enough mass), then ejecta in free expansion (EJ regime). Dump pure KE.

They have some comments:

- For ST regime, dumping thermal energy + redistribute mass is supposed to produce same results. However, they claim you must re-distribute mass within region to avoid overcooling, for M close to M_{sf} .
- EJ, ST regimes: redistribute mass/velocity/energy. For MC regime, do not reset gas properties, just add momentum to all zones.
- Argue Gatto+ 2015 feedback scheme is “marginal”, and that they needed to redistribute mass in injection sphere, and so they got too high SFR.
- Figure 6: the prescription looks kind of bonkers at high density, artifact of fixed (low) spatial resolution. This is the MC regime; discard thermal energy, just keep momentum.

Hopkins FIRE 2014MNRAS.445..581H

- Take $R_{\text{cool}} \approx 28 \text{ pc } E_{51}^{0.29} (n/1 \text{ cm}^{-3})^{-0.43} (Z/Z_{\odot} + 0.01)^{-0.18}$ the 0.01 bit surprises me, but I suspect it doesn't matter for our use case (solar metallicity...).
- When S-T phase is resolved, deposit correct amounts of thermal and kinetic energy (momentum) directly.
- When not, deposit momentum boosted by $\sim M_{\text{swept-up}}^{0.5}$, depending on how much mass got swept up by hot gas in the S-T stage.

$$\mathbf{p} = \mathbf{p}_{\text{ej}} \sqrt{\frac{M_{\text{ej}} + M_{\text{enc}}(< R_{\text{cool}})}{M_{\text{ej}}}}$$

and also decrease the thermal energy,

$$E_{\text{thermal,shocked}} \propto (R/R_{\text{cool}})^{-6.5}$$

which reduces its ability to do any more work.

- Broad idea makes sense, but I don't entirely follow the prescription.

13.5 Superbubble schemes and behavior

Kim Ostriker Ralaenu

Useful, recent discussion of sub-grid SNe schemes and winds by Chia-Yu Hu, see Section 4.3: [2019MNRAS.483.3363H](#)

14 Code development

Quoted from the ENZO manual: “If you change something, change the comments... Wrong comments are worse than no comments”.

Aaron’s list of proposed improvements!!! <https://bitbucket.org/aarontran/flash4.5-rad/issues>

Try to follow the FLASH idiom.

I (AT) recommend Google’s code style guides, less for specific actionables, more for philosophy / insight:

- Python style: <https://google.github.io/styleguide/pyguide.html>
- Documentation style: <https://developers.google.com/style/>

See in particular the “Parting Words” in the Python style guide.

Really useful tutorial presentations:

- FLASH center 2012: http://flash.uchicago.edu/site/flashcode/user_support/tutorial_talks/home.py?submit=RAL_May2012.txt
- collective wisdom – sfocu may be worth investigating for automated testing http://flash.uchicago.edu/site/flashcode/user_support/tutorial_talks/June2009/CollectiveWisdom.pdf

To overwrite a FLASH-default file, put stuff in Simulation/SimulationMain/YourProblem. Try to avoid putting in FLASH source if you can, it makes things messy.

Tips on FLASH architecture: how to add code in a SANE manner. http://flash.uchicago.edu/site/flashcode/user_support/tipsDoc.py?submit=tips_arch.txt

Do read (and maybe re-read) Chapter 4 of the FLASH manual before making large code changes.

“Maintenance cost is quadratic in number of features”: <https://statmodeling.stat.columbia.edu/2019/05/10/maintenance-cost-is-quadratic-in-the-number-of-features/#comments> To mitigate – be clear about what use cases are supported or not – reduce maintenance cost to linear (though for a tower of features, that also means code works only for one use case).

Idea: when you add a sub-unit with new! and! shiny! features, it’s easy to copy-paste a file and just add stuff to it. But, this is hard to maintain because you’re duplicating code:

```
Foo/
  Foo.F90
  Foo_bar.F90
  Foo_interface.F90
  FooMain/
    Foo.F90
    Foo_bar.F90
    FooLoremIpsum/
      Foo.F90      # contains lots of duplicate code
      Foo_bar.F90
```

Consider trying:

```
Foo/
  Foo.F90
  Foo_bar.F90
  Foo_lorem.F90
  Foo_ipsum.F90
  Foo_interface.F90
  FooMain/
```

```

Foo.F90          # modify to allow for LoremIpsum features
Foo_bar.F90
FooLoremIpsum/
    Foo_lorem.F90 # no duplication!
    Foo_ipsum.F90

```

Idea: don't over-constrain requirements in Config files (e.g. guardcell count); let user specify via setup call / setup shortcuts.

Config file directives: http://flash.uchicago.edu/site/flashcode/user_support/flash4_ug_4p5/node35.html#SECTION03252000000000000000

PARAMESH manual is no longer posted online, but available on the Wayback machine. Last snapshot was 2016 April 21. Visit https://web.archive.org/web/*/http://www.physics.drexel.edu/~olson/paramesh-doc/Users_manual/amr.html

Some thoughts on community code development: <https://extremecomputingtraining.anl.gov//files/2015/03/commcodesAnshu-aug8-830.pdf> <http://flash.uchicago.edu/cc2012/>

Git forking workflow: in your forked repo, do NOT press the SYNC button on bitbucket to grab upstream commits. It generates an extra merge commit, cluttering history. Maybe there's a workaround and I haven't discovered it. It's easier to do `git merge upstream/master`; so long as you didn't touch master (don't touch master!!) it will do a fast-forward merge and life will be good.

The FLASH center uses 3 space indents. No tabs, 2 space continuations(?). JW/JCIM/CB/MKRJ seem to use some mix of 2 space, 4 space, and tab.

14.1 Pull Requests

To submit a pull request, you'll generally want to make a branch, do some work, then push to your local repo.

We are following the Git forking workflow, see: <https://www.atlassian.com/git/tutorials/comparing-workflows/forking-workflow> and <https://gist.github.com/Chaser324/ce0505fbed06b947d962>

If possible, try to rebase to most recent commit before you submit PR.

For repo maintainers with write access: do `git remote prune origin` to clean now-removed remote branch pointers.

15 Lessons Learned, Cheatsheets, and FAQ

15.1 FLASH GOTCHAS!!!!

WARNING: sanitizing doesn't do anything apparently? All the fixing up comes from floors (smlrho, smallp, etc). Sources: Josh said so, and also <http://flash.uchicago.edu/pipermail/flash-users/2009-May/000467.html>

Some snags with USM and paramesh

- checkerboard B field structure with USM: <http://flash.uchicago.edu/pipermail/flash-users/2017-October/002427.html>
- spurious waves at refinement boundaries: <http://flash.uchicago.edu/pipermail/flash-users/2016-May/001956.html>
- more discussion about refinement boundary noise with usm... <http://flash.uchicago.edu/pipermail/flash-users/2019-March/002823.html>

Important change: in the file `Grid/GridMain/paramesh/interpolation/Paramesh4/prolong/Config`, comment out `PPDEFINE PM_OPTIMIZE_MORTONBND_FETCHLIST` to prevent weird segfaults. See the FLASH-USERS thread: <http://flash.uchicago.edu/pipermail/flash-users/2019-May/002901.html>

Important change: in FLASH4.5, building with parallel HDF5 1.10.5, you may hit following error:

```

io_h5_report_xfer_method.c: In function io_h5_report_xfer_method:
io_h5_report_xfer_method.c:105:26: error: H5D_MPIO_FILTERS undeclared (first use in this function)
    if (global_cause & H5D_MPIO_FILTERS) {

```

H5D_MPIO_FILTERS was removed??? in HDF5 1.10.2??? <https://github.com/hhoefflin/hdf5r/issues/97#issuecomment-4224>
That's weird. I would think ppl have built FLASH w/ new parallel HDF5.

RESOLUTION?? Directly modify:

source/IO/IOMain/hdf5/io_h5_report_xfer_method.c

HERE's the diff

```
104a105
> #if (H5_VERSION_LE(1,10,1))
107a109,113
> #else
>     if (global_cause & H5D_MPIO_NO_COLLECTIVE_MAX_CAUSE) {
>         ADD_CAUSE_TO_STR("H5D_MPIO_NO_COLLECTIVE_MAX_CAUSE ");
>     }
> #endif
```

FIXED!!!! :) :) :) NOTE this might be fixed in new FLASH... answer, YES, FLASH devs give the exact same resolution in FLASH 4.6.0. GOOD!

15.2 FLASH runtime parameters

15.3 FLASH ifdef variables

PE_HEAT indicates whether code tracks FUV photons or not; I (AT) changed behavior from toggling PE heating in general, to only toggling FUV photon heating from stars, because we need to be able to put in background PE heating separate from stars as well.

15.4 FLASH variable cheat sheet

15.4.1 FLASH field variables

First grouped by unit, then ordered alphabetically?

PHIO_VAR	photoionization rate [1/s],	ParticlesMain/ray_pe/MultiSourceSimple
PHHE_VAR	photoionization heating rate [erg/s],	ParticlesMain/ray_pe/MultiSourceSimple
PEFL_VAR	photoelectric flux [Habing units, scaled erg/s],	ParticlesMain/ray_pe/MultiSourceSimple
UVFL_VAR	scratch var for photoionizing flux [erg/s],	ParticlesMain/ray_pe/MultiSourceSimple
FUFL_VAR	scratch var for photoelectric flux [erg/s],	ParticlesMain/ray_pe/MultiSourceSimple

15.4.2 FLASH particle variables

Superbubble particles introduce several properties:

NSN_PART_PROP	number of SN in SB	ParticlesMain/sb
TCRT_PART_PROP	creation time of SB	ParticlesMain/sb
TSN_PART_PROP	time interval between SNe	ParticlesMain/sb
SBID_PART_PROP	SB unique identifier	ParticlesMain/sb

In the supernova unit code, tracer particles overload some particle properties in order to grab information about the surrounding gas. The mesh-to-particle map happens in `ParticlesMapping/Particles_mapFromMesh.F90`; the overloaded properties are:

NSN_PART_PROP	maps to grid's TEMP_VAR	ParticlesInitialization/pt_setDataStructures
MASS_PART_PROP	maps to grid's DENS_VAR	ParticlesInitialization/pt_setDataStructures

OK stuff for radiation and amuse coupling

EION_PART_PROP	energy of ionizing photons
NION_PART_PROP	number of ionizing photons
SIGH_PART_PROP	gas cross-section to ionizing radiation

EPEP_PART_PROP	Average energy of PE photon
NPEP_PART_PROP	number of PE photons
SIGD_PART_PROP	dust cross-section to PE photons

Stuff for winds coupled to AMUSE

DMDT_PART_PROP	wind mass loss rate
VELW_PART_PROP	wind mass loss rate

15.4.3 FLASH species variables

15.5 Frequently asked questions

- **Question:** How long does a run take?

Answer: on Columbia University’s cluster “Habanero” (Intel E5-2650v4, 2.2GHz; 5.3 GB/core; 24 cores/node; EDR Infiniband), with 6 refinement levels, block size 16^3 , and 186 cores, evolving JUST GAS (no N-body, no feedback) to 3 Myr took 10 days wall clock time or $\sim 50,000$ CPU-hours. This was a very crude test run, not sure if I used bridge code or plain FLASH. Not using parallel HDF5. A production run will take 100,000s of CPU-hours.

- **Question:** How do I add a new runtime parameter?

Answer:

1. define in `Simulation_data.F90`
2. define and document in `Config`
3. in `Simulation_init.F90`, call `RuntimeParameters_get(...)`
4. add to your `flash.par`

- **Question:** How do I track down a variable or error message?

Answer: in your object directory, do something like `grep -rInt -I "INSERT ERROR MESSAGE" *`

- **Question:** How do I decipher FLASH units, runtime parameters, and setup calls?

Answer: look at `bin/setup_shortcuts.txt`, `object/setup_call`, `object/setup_params`, `object/setup_units`.

- **Question:** How much memory does FLASH use?

Answer: estimate as:

$$\begin{aligned}
 &8 \text{ bytes} \times (\text{NUNK_VARS} + \text{NFACEVAR} + \text{NFLUXVAR}) \\
 &\quad \times (\text{NXB} + 2 \times \text{GUARDCELLS}) (\text{NYB} + 2 \times \text{GUARDCELLS}) (\text{NZB} + 2 \times \text{GUARDCELLS}) \\
 &\quad \times \text{MAXBLOCKS}
 \end{aligned}$$

These variables are defined by `Flash.h`, in the object directory. For the simulation `Cube` with unsplit MHD solver, 16^3 blocks, and `MAXBLOCKS=300`, the memory usage is:

$$8 \text{ bytes} \times (46 + 2 + 12) \times (16 + 12)^3 \times 300 \approx 3.2 \text{ GB}$$

- **Question:** with FLASH4.2.2, how do I compile with large values of MAXBLOCKS? Setup works, but make fails.

Answer: in `Makefile.h`, request the compiler flag `-mmodel=medium`. Search flash-users for “mmodel” to learn more.

- **Question:** “needed full blk... but could not find it or only found part of it in the message buffer. Contact PARAMESH developers for help.”

Answer: <http://flash.uchicago.edu/pipermail/flash-users/2015-February/001637.html> See also <http://flash.uchicago.edu/pipermail/flash-users/2017-April/002262.html>

- **Question:** when restarting, what numbers should I use for checkpoints and plotfiles?

Answer: checkpoints, use the files you want to restart from. Plot files, use the next plot file numbers to create.

- **Question:** FLASH performance?

Answer: Mordecai's proposal quotes 150e-6 core-seconds per zone update. I (AT) measured very similar on Habanero cluster at Columbia, which is Intel Xeon E5-2650v4.

Sean Couch reports on FLASH users list (November 2014) that TACC Stampede (Intel) gave 1e-8 core-hours per zone update, with near-perfect weak scaling up to 4096 cores. Translation is 36e-6 core-seconds per zone. So, our version of FLASH w/ all bells+whistles is about 4x slower than vanilla!!!! Not unexpected w/ radiative transfer, detailed spatially varying heat/cool, etc., but still not amazing.

- **Question:** when to do EOS update? How?

Partial answer: <http://flash.uchicago.edu/pipermail/flash-users/2017-May/002336.html>